

KleverOne Planning Agent — Sprint 1 Submission

Five user stories · sprint scope · three questions before more is written

The planning agent's job is to produce a first-draft media plan a Strategist will actually use, with SSP-direct as the default, in a way that breaks the DSP-default habit. The output must make the SSP-direct-first / DSP-as-exception logic *visible and defensible* — not just true in the prompt. Sprint 1 ships the slice that validates that thesis end-to-end, plus the regression suite that protects it from week-two prompt regressions.

User stories with acceptance criteria

1 Brief intake and confirmation

As a Strategist, I paste a brief in plain language and confirm what the agent extracted before it drafts the plan.

- Free text in → structured extraction within 60s
- Confirmation shows the 11 fields (advertiser, vertical, budget, flight, KPI, goal, geo, audience, inventory, exclusions, channels)
- All fields inline-editable; edits captured as structured events
- Low-confidence extractions marked "needs review" and must be confirmed before Generate

2 SSP-direct-first plan, defended DSP gate

As a Strategist, I get a draft plan in which SSP-direct is the default and any DSP line carries a justification I can defend to a client.

- Briefs without DSP-eligible signals yield **zero** DSP lines
- Every DSP line carries a justification: what's unique, what SSP/AdCP cannot do, why this share
- Plan-level DSP gate summary: considered, earned, rejected with reasons
- All `capability_id` and `deal_ref` values resolve; hallucinations fail the generation
- Line shares sum to 100% of budget

3 Allocation reasoning that defends itself

As a Strategist, every line tells me why it got the share it did; editing a DSP line above its ceiling surfaces the original reasoning inline.

- Every line has an `allocation_rationale` distinct from `rationale`
- Allocation rationale names: the share, the brief constraint, a trade-off vs an alternative share
- DSP lines carry a `share_ceiling` + reason
- Editing above ceiling surfaces the original reason in the row, not tooltip-only

4 Edit, capture, hand off

As a Strategist, I edit the plan inline, edits become structured feedback, and I can hand off to AdOps in a usable format.

- Spend %, rationale, allocation rationale all inline-editable; totals recalculate
- "N edits captured" indicator whenever the plan diverges from the original
- CSV export with stable column order for AdOps trafficking
- Markdown copy for Slides/Notion paste; plans persist per user via stable URL

5 Regression-protected DSP default (team story)

As the planning team, we detect DSP-default creep before a prompt change or model swap reaches a Strategist.

- Regression set of ≥ 15 briefs across pure-SSP, YouTube-in-scope, retargeting-performance, retail-performance, and underspecified-edge cases
- Suite runs on every prompt/model change in CI; results versioned
- **Hard gates** (block deploy): schema 100%, zero capability/deal hallucinations, DSP share $\leq 15\%$ on pure-SSP briefs, every DSP line carries a non-empty justification
- **Soft gates** (LLM-judged vs reference plans): rationale, allocation, and justification quality — regressions reviewed before merge

Sprint 1 vs deferred

SHIPS IN THE 2-WEEK SPRINT

- Brief intake + confirmation with edit telemetry
- Plan generation: deterministic DSP gate + LLM rationale and allocation reasoning
- Structured output with required `dsp_justification` on DSP lines
- Inline editing on spend, rationale, allocation rationale; structured edit capture
- CSV and markdown export
- Regression suite: ≥15 golden briefs, deterministic checks, LLM-judge rubric, CI
- Per-user plan persistence on Vercel Postgres
- Auth via Vercel-protected route + email allowlist
- Sentry, prompt versioning, call logs

CUT FROM V1

- Live SSP/DSP MCP calls during generation — use daily-refreshed deal snapshot; live calls are a *buying* agent concern
- AdCP integration — stub publisher-direct as named placeholders if not MCP-shaped yet
- Ad-server line export to GAM — CSV is enough for v1
- Catalog admin UI — JSON in repo with PR-flow updates
- Brand identity polish — placeholder chrome until assets land
- History sidebar and shareable URLs beyond per-user
- Low-confidence flagging on extracted fields — needs model-side confidence the v1 extractor won't emit cleanly

Why this cut. The thesis is *an agent producing first-draft plans Strategists will use, with SSP-direct-first reasoning visible and defended against regression* — that thesis ships only if the eval harness ships with it, otherwise the cultural mechanic regresses on the first prompt tweak and we ship a demo, not a product. LLM-assisted development is what lets us put the eval suite *in* sprint 1 rather than punting it; the velocity gain on code is what buys the discipline to ship the regression layer that normally gets cut.

Top 3 questions before writing another line of this brief

1 Who is the named authority on "this is a good plan"?

The eval suite and LLM-judge prompts have to calibrate against a human's judgement. Without one committed Strategist or planning lead signing off on the golden brief set and the rubric — available ~4 hrs/week through sprint 1 — the eval drifts and the cultural mechanic loses its anchor.

2 Is the capabilities catalog fit for purpose today, and who owns its content?

Description, channels, KPIs served, limitations — is every entry populated, accurate, current? If not, sprint 1 starts with content cleanup, outside the two-engineer scope. We need the catalog owner named and the current state confirmed before locking the prompt and the eval.

3 What does "ad server source of truth" mean concretely for the plan-output schema?

v1 doesn't write to the ad server, but the plan schema has to remain translatable to GAM's (or equivalent) insertion-order / line-item shape — otherwise the buying agent inherits a translation layer that didn't have to exist. We need that shape before locking the v1 plan schema.